IMEN574 / IDSC574 · PROGRAMMING FOR AX

15

PyTorch

Sangwoo Mo · POSTECH
Spring 2027

Where We Are

01 **Programming with AI**

Lectures 2–4

02 **Python and Classical Machine Learning**

Lectures 5–13

03 **Deep Learning and Foundation Models**

Lectures 14–22

Outline: PyTorch

01 **Tensor Contracts**

Construct inputs and state

02 **One Learning Step**

Trace gradients and updates

03 **A Complete Run**

Validate and retain state

01 · Tensor Contracts

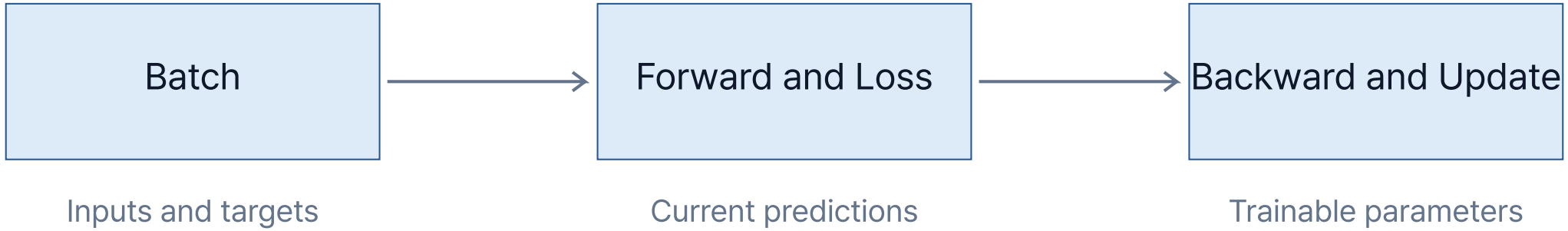
01 **Tensor Contracts**
Construct inputs and state

02 **One Learning Step**
Trace gradients and updates

03 **A Complete Run**
Validate and retain state

The Training Loop

A small end-to-end computation organizes the framework



The framework connects a repeated computation with state updates

Tensor Values and Shapes

A tensor stores numbers with an explicit axis contract

	Feature 1	Feature 2
Sample 1	1	2
Sample 2	3	4
Sample 3	5	6

```
X.shape = (3, 2)
```

Rows are observations and columns are features in this example

Dtype Contracts

Input values and target indices have different roles

Value	Typical Dtype	Role
Features	float32	Differentiable arithmetic
Class target	int64	Class index
Mask	bool	Select valid positions

A class index is not a floating-point probability

Tensor Construction

Create a small CPU tensor without downloading data

```
import torch
X = torch.tensor([[1., 2.], [3., 4.]])
y = torch.tensor([[3.], [7.]])
assert X.shape == (2, 2)
assert y.shape == (2, 1)
```

The example has two observations and two features

Batch Matrix Multiplication

A batch shares one set of parameters

	X	w	X @ w
Shapes	B × d	d × 1	B × 1

```
pred = X @ w + b
```

One weight vector produces one output per observation

Caveat: Silent Broadcasting

Compatible shapes can produce the wrong error matrix

Tensor	Shape
pred	(B, 1)
target	(B,)
difference	(B, B)

A column minus a vector can compare every pair

Broadcasting With NumPy

Verify the unintended shape before moving to torch

```
pred=np.array([[1.],[2.],[3.]])
target=np.array([1.,2.,3.])
print("wrong",(pred-target).shape)
fixed=target[:,None]
print("correct",(pred-fixed).shape)
assert np.allclose(pred,fixed)
```

```
wrong (3, 3)
correct (3, 1)
```

Demo: Paired and All-Pairs Loss

Why is a perfect fit nonzero with a flat target? Which residual shape matches the task?

Target shape (6,1): paired w_1 2 w_2 3 bias 1

Row	Prediction	Target	Residual
1	1	1	0
2	4	4	0
3	3	3	0
4	6	6	0
5	8	8	0
6	9	9	0

Explicit broadcasting arithmetic | target (6,1); residual shape 6×1
Displayed loss 0 | correct paired MSE 0
The same perfectly fitted predictions can have a nonzero all-pairs loss

Manual broadcasting arithmetic, not a PyTorch runtime

Answer: A Perfect Fit and the Wrong Loss

Only the paired target produces one residual per observation

Target Shape	Residual Shape	MSE
(6,1)	(6,1)	0
(6,)	(6,6)	15.611111

A successful tensor operation can compute the wrong statistical quantity

Caveat: Loss Shape Is a Task Contract

An all-pairs loss can pull correct predictions away from their own target

Check	Expected
Observation axis	Six examples
Prediction and target	Same 6×1 shape
Residual	One value per example

Inspect the residual before trusting the loss number

Parameters and Buffers

Only selected tensors are optimized

State	Optimizer Parameter
Weight	Yes
Bias	Yes
Running statistic	No
Input batch	No

Model state includes values that are not trainable parameters

Model Initialization

Create parameters once before the training loop



One initial state

Same parameters

Repeated updates

Reinitializing inside the loop discards learned progress

Device Consistency

Keep interacting tensors on a compatible device

Object	Device
Model	CPU
Input	CPU
Target	CPU

The complete example runs on CPU

A Deterministic Starting Point

Record seeds while keeping reproducibility limits explicit

```
import torch
torch.manual_seed(7)
model = torch.nn.Linear(2, 1)
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)
```

The model and optimizer are created before any update

02 · One Learning Step

01 Tensor Contracts

Construct inputs and state

02 One Learning Step

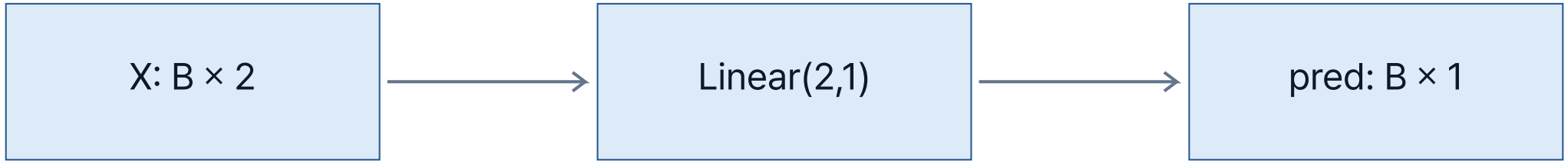
Trace gradients and updates

03 A Complete Run

Validate and retain state

Forward Pass

Apply the current model to one batch



Observed features

Same weights

One value per row

Forward evaluation reads the current parameters

A Minimal Module

Put the forward computation inside nn.Module

```
class TinyModel(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = torch.nn.Linear(2, 1)
    def forward(self, x):
        return self.linear(x)
```

Registered submodules expose parameters to the optimizer

Loss as a Scalar

Reduce per-example errors into one optimization objective

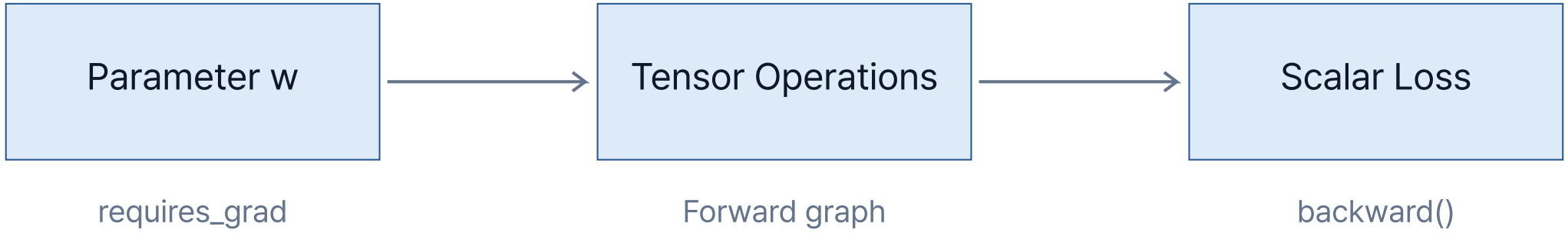
	Pred	Target	Squared Error
Sample 1	1	2	1
Sample 2	3	2	1

$$\text{MSE} = \text{mean}((\text{pred} - \text{target})^2) = 1$$

The mean loss is one despite opposite error directions

Autograd Dependencies

Record operations on tensors that require gradients



Autograd follows recorded dependencies to trainable leaves

Backward Pass

Populate gradients without updating parameters

```
optimizer.zero_grad()
pred = model(X)
loss = torch.nn.functional.mse_loss(pred, y)
loss.backward()
for p in model.parameters():
    assert p.grad is not None
```

Calling backward fills gradient fields rather than changing weights

Gradient Accumulation in Code

Repeated backward calls add gradients by default

Operation	New Gradient	Stored Gradient
First backward	2	2
Second backward	2	4
zero_grad(set_to_none=True)	None	None

A second backward can double an uncleared gradient

https://docs.pytorch.org/docs/stable/generated/torch.optim.Optimizer.zero_grad.html

Optimizer Step

Apply the stored gradients to the referenced parameters



p.grad

SGD or another rule

Next forward uses them

The update rule operates after the gradients are available

One Complete Update

Keep the five operations in a visible order

```
optimizer.zero_grad()
pred = model(X)
loss = loss_fn(pred, y)
loss.backward()
optimizer.step()
```

Clear, predict, score, differentiate, then update

A Shared Six-Row Batch

Compute the first batch gradient before using the tracer

x_1	x_2	Target	Initial Prediction
0	0	1	0
0	1	4	0
1	0	3	0
1	1	6	0
2	1	8	0
1	2	9	0

The initial mean squared loss is 34.5

Demo: Forward, Backward, and Step

What changes after Backward? What happens if two backward passes run without Zero grads?

Learning rate 0.05 Batch Six fixed rows Forward Backward Step Zero grads Reset

Parameter	Current Value	Stored Gradient
w_1	0	0
w_2	0	0
bias	0	0

Batch 6×2 → prediction 6×1 → mean squared loss

Manual gradient tracer, not PyTorch | Forward ready; forward cache ready
Current MSE 34.50; backward accumulations 0, parameter steps 0
Forward → Backward → Step; gradients persist until Zero grads is called

Manual linear-layer chain rule; PyTorch is not executed here

Answer: First Batch Gradient

Backward fills buffers and Step changes the same parameters

State	w_1	w_2	bias
Initial	0	0	0
One backward	g = −11.333333	g = −12	g = −10.333333
Step, η=0.05	0.566667	0.600000	0.516667

The parameter update uses the stored batch gradients

Caveat: Accumulated Gradients

A second backward can double the next update if buffers are not cleared

Sequence	Gradient Buffer	Weights Changed?
Backward	g	No
Forward → Backward	2g	No
Zero grads	0	No
Backward → Step	Depends on fresh forward	Yes if gradient nonzero

Clearing buffers and updating weights are separate operations

03 · A Complete Run

01 Tensor Contracts

Construct inputs and state

02 One Learning Step

Trace gradients and updates

03 **A Complete Run**

Validate and retain state

Epochs and Batches

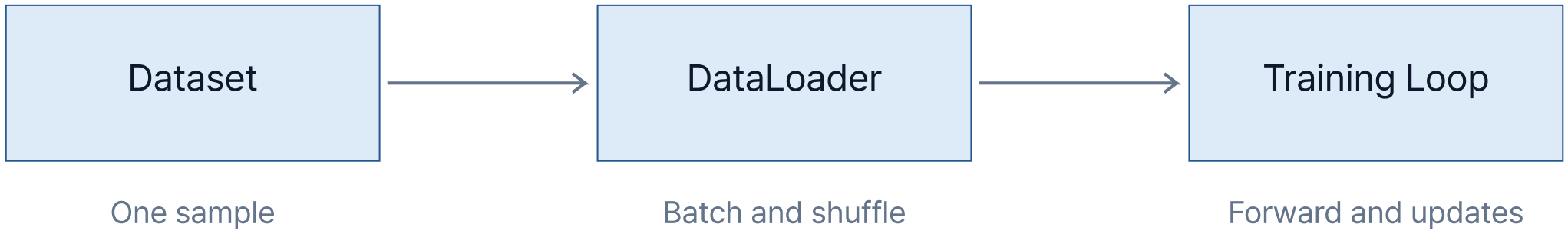
Separate passes through data from individual updates

Rows	Batch Size	Steps / Epoch
12	4	3
12	6	2
12	12	1

A larger batch yields fewer updates per epoch

DataLoader Responsibilities

Produce batches while leaving training logic explicit



The loader handles sampling rather than model optimization

Training and Evaluation Modes

Mode changes affect selected layer behavior

Layer	train()	eval()
Dropout	Random masking	Disabled
BatchNorm	Batch statistics	Stored statistics

Evaluation mode changes layer behavior independently of gradients

Validation Without Gradients

Measure fixed parameters without recording a training graph

```
model.eval()
with torch.no_grad():
    pred = model(X_valid)
    valid_loss = loss_fn(pred, y_valid)
model.train()
```

Validation reads a frozen model on held-out inputs

Metric Aggregation

Weight batch means by the number of examples

	Rows	Mean Loss	Loss Sum
Batch A	4	1	4
Batch B	2	4	8
Total	6		12 / 6 = 2

A mean of batch means would incorrectly give 2.5

Checkpoint State

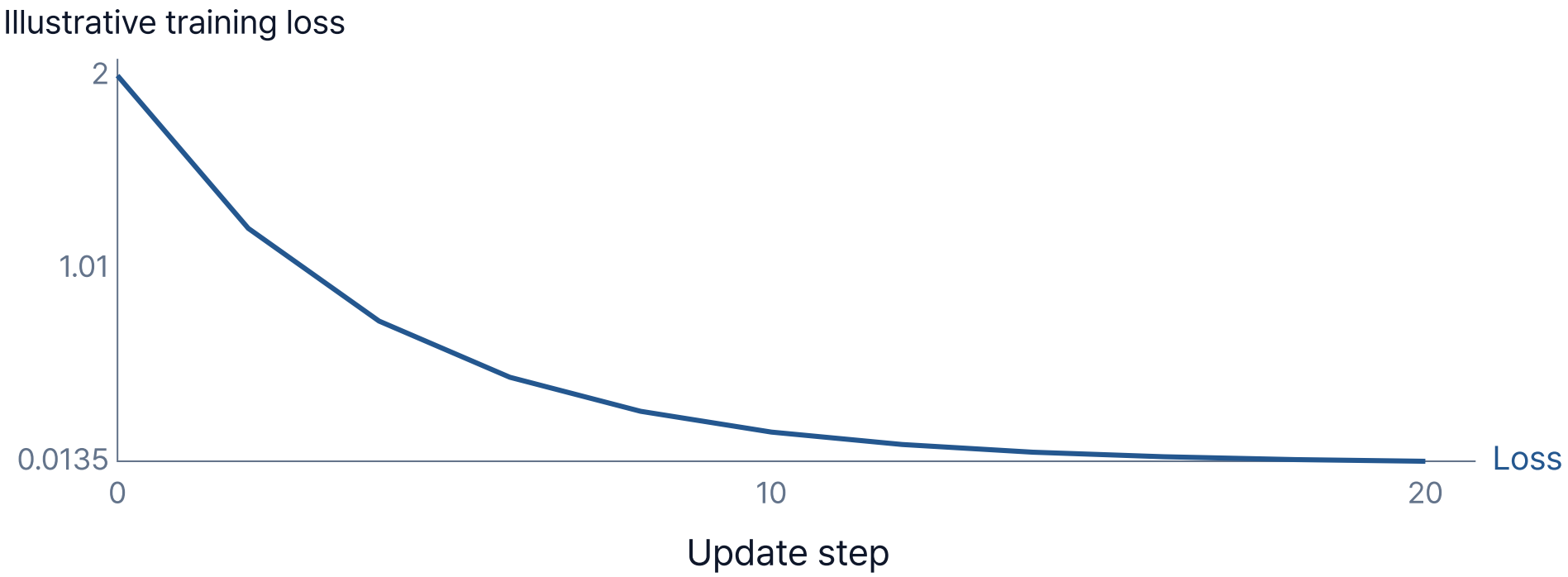
Retain the model state needed for the intended restart

Purpose	State
Inference	Model parameters and buffers
Resume training	Model + optimizer + step
Reproduce selection	Validation protocol and configuration

A saved model alone may not resume the same optimization state

Tiny-Batch Debugging

First fit a few fixed examples without pipeline complexity



A tiny fixed batch isolates basic fitting failures

Gradient and Shape Checks

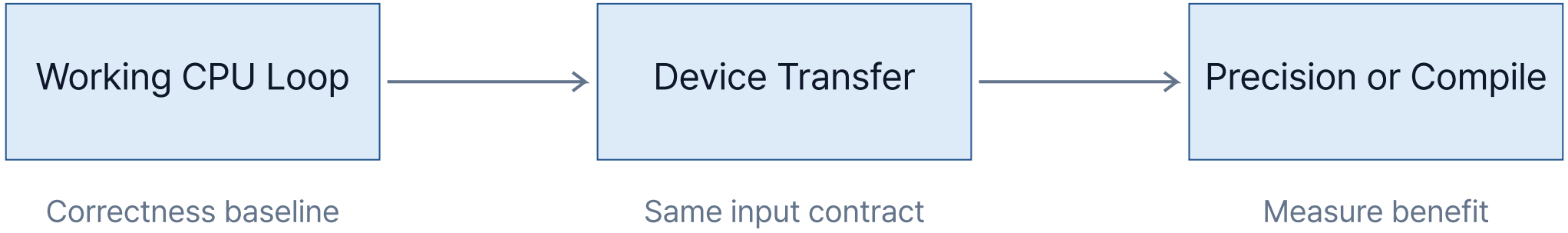
Inspect the nearest failed contract first

Symptom	First Check
No learning	Missing gradient or step
Exploding loss	Scale and learning rate
Wrong dimensions	Batch and target shapes
Bad validation	eval mode and split

A concrete failed contract is more useful than changing the entire model

Optional Acceleration

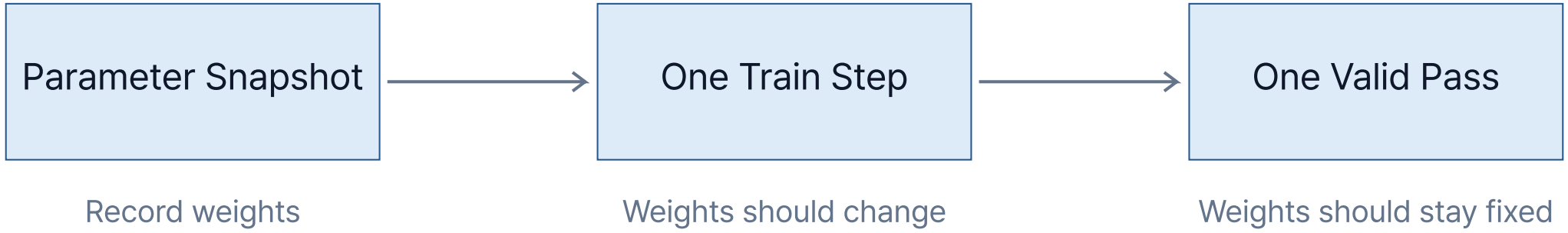
Add device and precision changes after the CPU loop works



Acceleration comes after a reproducible reference computation

Auditing AI-Written Training Code

Trace what changes during one step and one validation pass



Parameter changes distinguish training from measurement

Takeaway: PyTorch

01 Keep tensor meaning and shapes explicit

02 Backward computes gradients and step updates parameters

03 Validation changes neither parameters nor the evaluation boundary